

AI image detector — team plan

Sep 25, 2026 · @SIVA

We detect AI-generated images by training a small classifier on top of a frozen vision foundation model, scored only on generators it has never seen, with a provenance check (C2PA, EXIF, PNG chunks) running first.

The method

Take a pretrained vision foundation model (DINOv2 or CLIP), keep it frozen, and train a small classifier on its image embeddings, using real and AI images matched in format, size and content, with heavy compression and resize augmentation.

Each part earns its place:

- **Foundation model, not a CNN from scratch.** It already knows what real images look like, so it learns "unnatural" in a general way instead of memorising one generator's quirks.
- **Matched data.** Without it the model learns shortcuts like "PNG means fake" and nothing about AI.
- **Degradation augmentation.** Real images arrive compressed, resized and screenshotted, and the signal has to survive that.
- **Scored on unseen generators.** Train on Stable Diffusion, test on Midjourney or Flux. In-distribution accuracy hits 99% and means nothing.
- **Provenance first.** C2PA credentials, EXIF/XMP fields and PNG text chunks are near-proof when present, so they override the classifier. The model covers the majority of images, where that metadata was stripped.

Build order

One change at a time, each compared against the step-2 baseline on the same held-out generator.

1. Extract DINOv2 embeddings for every image once and save them to disk.
2. Train logistic regression on the embeddings and measure AUC on a held-out generator.
This is the baseline.
3. Add augmentation (JPEG 30–100, resize 0.5–1.5x, crop) and re-measure.
4. Compare backbones: DINOv2, CLIP, SigLIP, and a ViT-L against a ViT-B.
5. Switch from a frozen backbone to LoRA fine-tuning of the last few blocks.
6. Add the reconstruction trick: real images passed through a diffusion VAE, labelled AI.

7. Add a low-level branch (high-pass or frequency features) and fuse it with the backbone.

Steps 1 and 2 run on a free Colab T4. Step 5 onward needs a rented GPU, roughly 10–50 per training run.

Who does what

Role	Owns	Main tasks	Skills to learn
Model lead	The model and training code	Steps 1–7 above; picks checkpoints on held-out-generator scores only	PyTorch, transformers , timm , peft (LoRA), Weights & Biases
Data lead	Every image that goes into training	Collect real photos in the target distribution; generate AI images across many models and settings; caption real images and generate matched AI versions; VAE reconstruction set; normalise formats and sizes; track filename to generator	Python scripting, diffusers , Pillow/OpenCV, pandas
Evaluation lead	The scoreboard, and being the skeptic	Freeze held-out-generator test sets; degradation suite (JPEG 30/50/75, resize, crop, screenshot); real-photo stress set (edited, filtered, screenshots); report AUC and TPR at 1% FPR per generator and per degradation; hunt shortcut learning	scikit-learn metrics, pandas, matplotlib, experiment design
Product lead	Turning a checkpoint into something usable	API (image in, score plus which signals fired); provenance layer before the model; Gradio demo; deployment, batching, speed; bring real customer images back to Data	FastAPI, Docker, Modal or RunPod, Gradio, later TypeScript/React

Everyone first learns the shared basics: Python, PyTorch fundamentals, git and pull requests, the Linux command line and remote GPUs, and train/validation/test splits with precision, recall and AUC. A good starting point is the PyTorch "Learn the Basics" tutorials or fast.ai's Practical Deep Learning.

Handoffs

- Data gives Model a folder per source plus a manifest of filename to generator.
- Model gives Evaluation checkpoints in an agreed format; Evaluation never uses Model's training or scoring code, or the check is not independent.
- Evaluation's findings go back to Data ("we need more filtered selfies") and to Model ("it collapses at JPEG 50").
- Model gives Product a stable loading interface so checkpoints swap without Model's involvement.
- One weekly meeting: Evaluation presents the results table, the team picks the single biggest problem to fix next.

Pitfalls

- **A near-perfect first score.** If AUC comes out around 0.99 on the first run, suspect a shortcut: real images all JPEG and AI all PNG, different resolutions, or different subject matter. Re-save everything through one pipeline and try again.
- **Selecting checkpoints on in-distribution validation.** Always pick on held-out generators.
- **Reporting accuracy alone.** Report TPR at 1% FPR too. False positives on real photographers' work are what destroy trust.
- **Resizing whole images to 224px.** That erases the artifacts. Take crops at native resolution and average scores over several crops at inference.
- **Full fine-tuning too early.** It overfits to the generators in the training set.
- **Never looking at the failures.** Twenty misclassified images teach more than any accuracy number.

First month

Week	Everyone	Data	Model	Evaluation	Product
------	----------	------	-------	------------	---------

1	Shared basics; each person trains one toy image classifier end to end	Survey sources; pull a slice of GenImage	Same	Same	Set up repo, environment, GPU access
2	—	First matched set, about 10k images	Frozen-DINOv2 baseline running on public data	Freeze the first held-out-generator test set	Skeleton API plus C2PA/EXIF/PNG-c hunk checks
3	—	Add two more generators; VAE reconstruction set	Steps 2–3: baseline number, then augmentation	Degradation suite; first results table	Serve a dummy checkpoint through Gradio
4	Weekly results meeting becomes routine	Fill the gaps Evaluation found	Step 4: backbone comparison	Real-photo stress set	Show the demo to 3–5 potential users

The month is done when there is one number everyone trusts: AUC on a generator the model has never seen, measured after JPEG compression.